

1 Theory

Half Adder

A half adder is a digital circuit that adds two single-bit binary numbers and produces a sum and a carry.

Truth Table

<i>A</i>	<i>B</i>	<i>Sum</i>	<i>C_{out}</i>
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Boolean Expression

$$Sum = A \text{ xor } B$$

$$C_{out} = A \text{ and } B$$

Full Adder

A full adder is a digital circuit that adds three single-bit binary numbers (two input bits and a carry bit) and produces a sum and a carry.

Truth Table

<i>A</i>	<i>B</i>	<i>C_{in}</i>	<i>Sum</i>	<i>C_{out}</i>
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Boolean Expression

$$Sum = A \text{ xor } B \text{ xor } C_{in}$$

$$C_{out} = (A \text{ and } B) \text{ or } (B \text{ and } C_{in}) \text{ or } (C_{in} \text{ and } A)$$

Half Subtractor

A half subtractor is a digital circuit that subtracts two single-bit binary numbers and produces a difference and a borrow.

Truth Table

A	B	Dif	B_{out}
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Boolean Expression

$$Dif = A \text{ xor } B$$
$$B_{out} = \text{not}(A) \text{ and } B$$

Full Subtractor

A full subtractor is a digital circuit that subtracts three binary bits (two input bits and a borrow bit) and produces a difference and a borrow.

Truth Table

A	B	C_{in}	Dif	B_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	0
1	1	1	1	1

Boolean Expression

$$Dif = A \text{ xor } B \text{ xor } C_{in}$$
$$C_{out} = (\text{not}(A) \text{ and } C_{in}) \text{ or } (\text{not}(A) \text{ and } B) \text{ or } (B \text{ and } C_{in})$$

2 Implementation and Observation

2.1 Half Adder

VHDL code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
-- designing half adder circuit
entity Half_Adder is
    Port ( A : in STD_LOGIC;
          B : in STD_LOGIC;
          Sum : out STD_LOGIC;
          Cout : out STD_LOGIC);
end Half_Adder;

architecture Behavioral of Half_Adder is

begin
    Diff <= A xor B;
    Bout <= A and B;

end Behavioral;
```

RTL Schematic Diagram

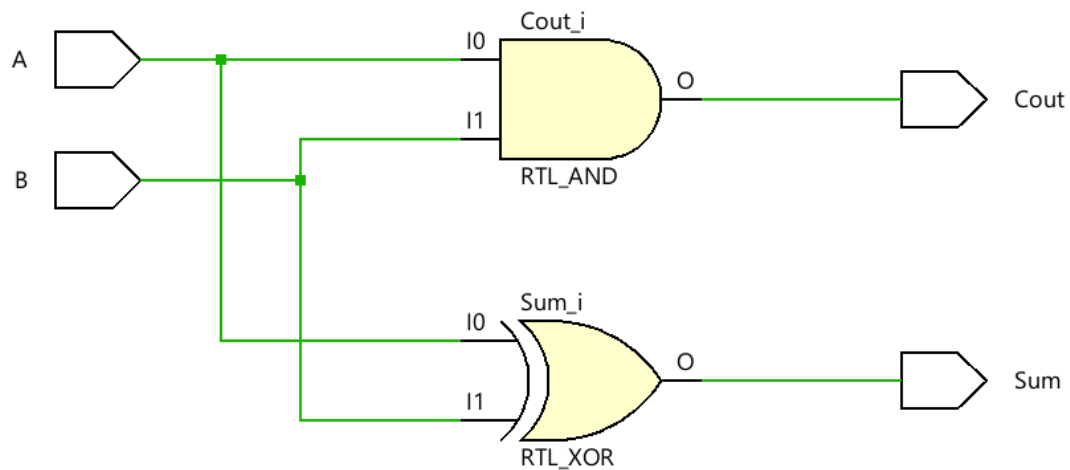


Figure 1: RTL Schematic Diagram of half adder

RTL design is used to describe the behavior of the half adder as shown in figure 1

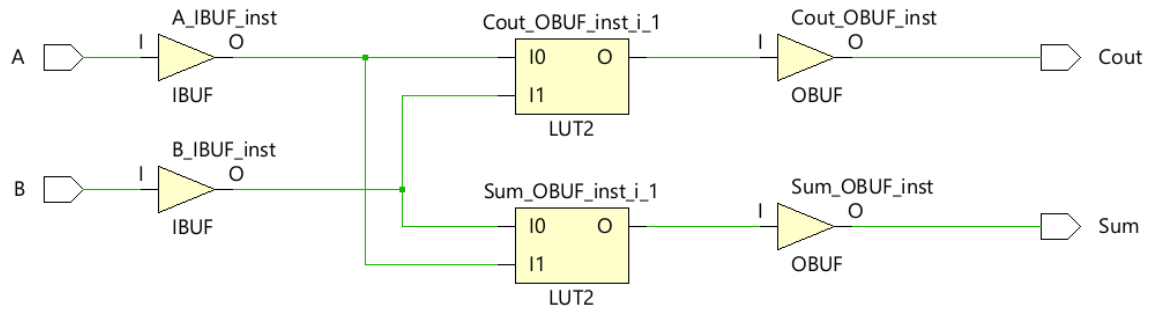


Figure 2: Synthesis Schematic Diagram of half adder

Synthesis Schematic Diagram

Schematic design is used to describe the implementation of the half adder circuit as shown in figure 2

Timing diagram

Timing diagram test is done to check whether the VHDL program and schematic diagram is valid for the given truth table of half adder.



Figure 3: Timing Diagram of half adder

Device Utilization

IO (Input/Output) refers to the physical FPGA pins used for communication with external devices, tracking how many of these pins are utilized for inputs and outputs. Utilization, on the other hand, measures the internal FPGA resource consumption, such as lookup tables (LUTs), flip-flops, block RAM (BRAM), and DSP slices. Together, these concepts provide a comprehensive view of FPGA design efficiency, as shown in figure 4.

Power Estimation

Figure 5 shows estimation of static power (no change in input and output), dynamic power (power required when VHDL program is burned in FPGA) and other related power data of a half adder circuit.

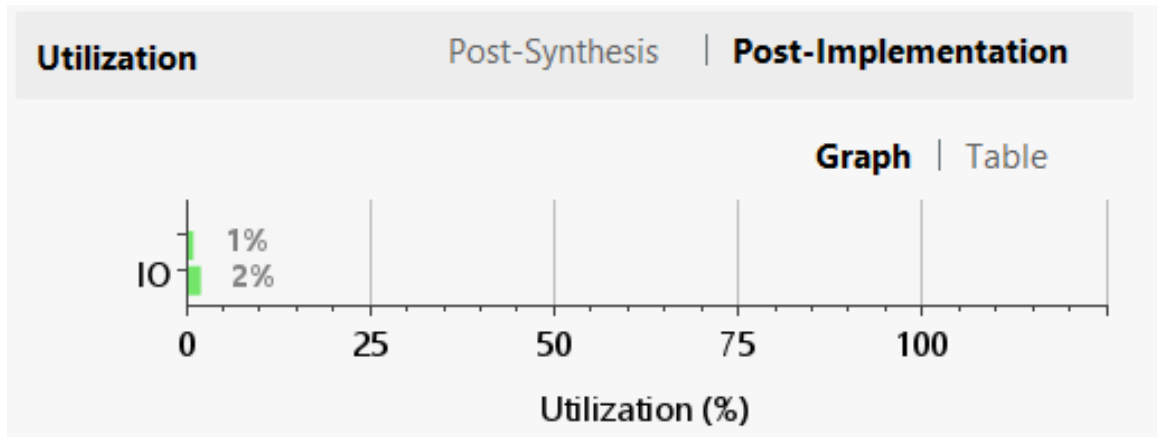


Figure 4: Device Utilization of half adder

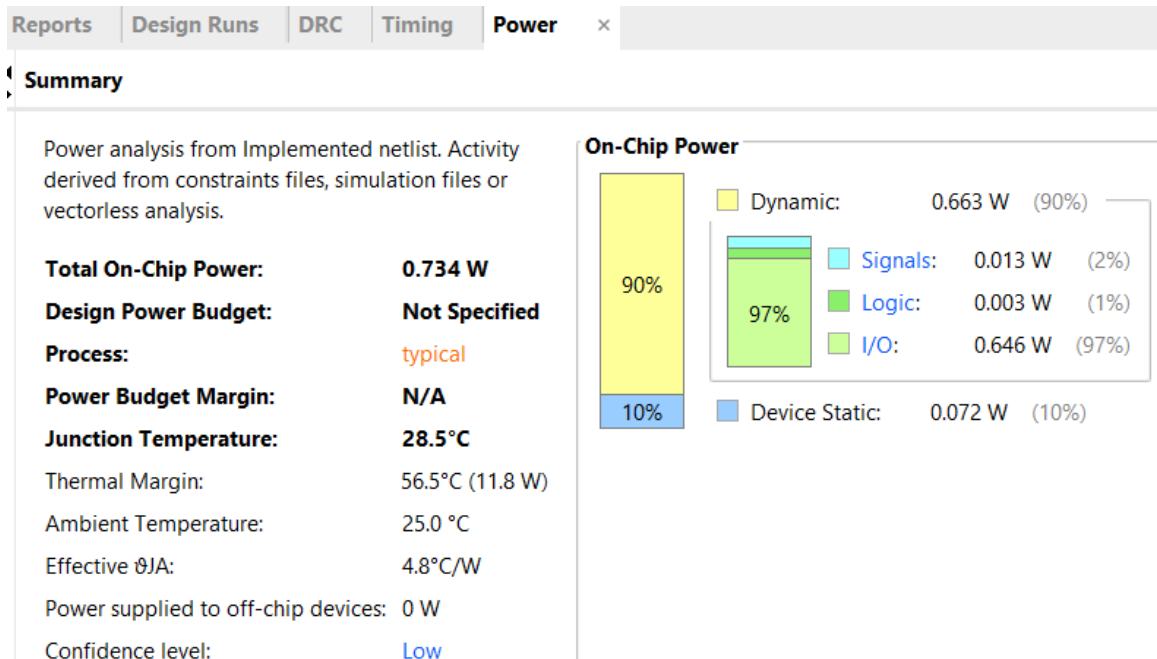


Figure 5: Power Estimation of half adder

2.2 Full Adder

VHDL code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

--Full Adder

entity Full_adder is
    Port ( A : in STD_LOGIC;
          B : in STD_LOGIC;
          Cin : in STD_LOGIC;
          Sum : out STD_LOGIC;
          Cout : out STD_LOGIC);
end Full_adder;

architecture Behavioral of Full_adder is

begin
    Sum <= A xor B xor Cin;
    Cout <= (A and B) or (B and Cin) or (Cin and A)
end Behavioral;
```

RTL Schematic Diagram

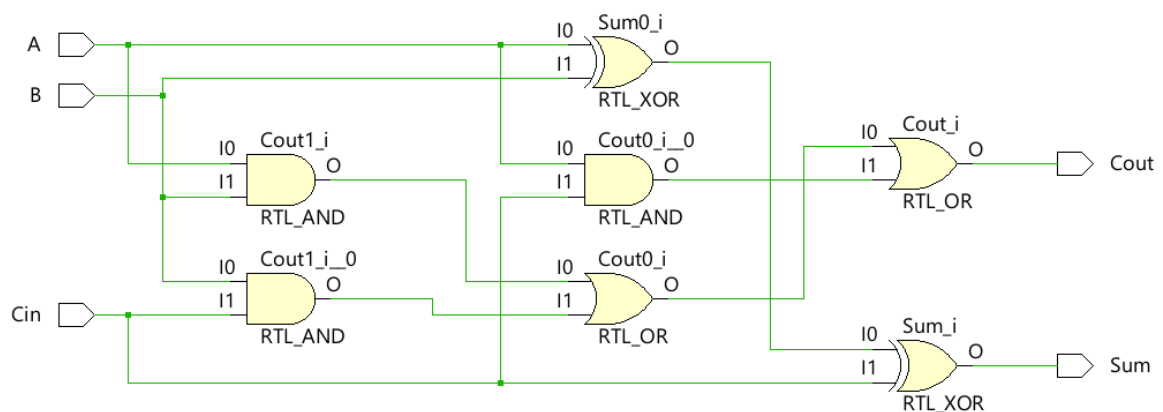


Figure 6: RTL Schematic Diagram of full adder

RTL design is used to describe the behavior of the full adder as shown in figure 6

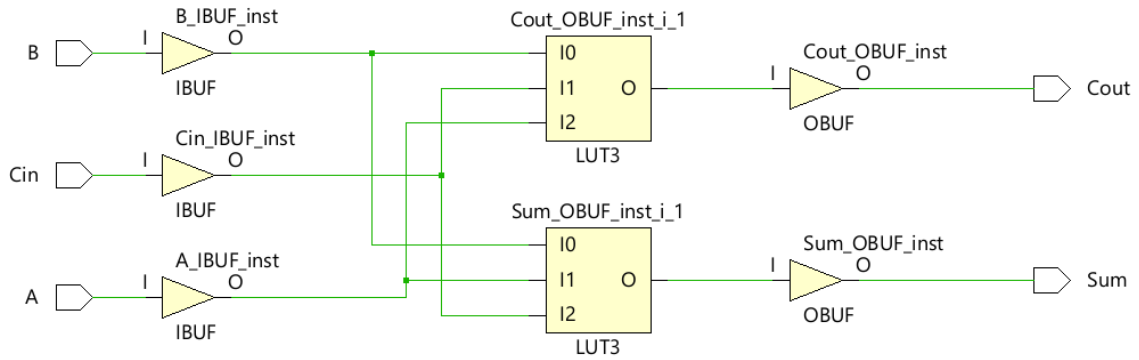


Figure 7: Synthesis Schematic Diagram of full adder

Synthesis Schematic Diagram

Schematic design is used to describe the implementation of the full adder circuit as shown in figure 7.

Timing diagram

Timing diagram test is done to check whether the VHDL program and schematic diagram is valid for the given truth table of full adder.

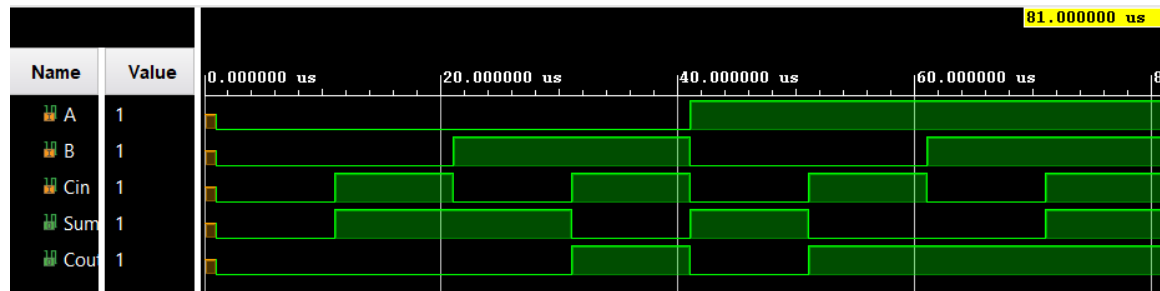


Figure 8: Timing Diagram of full adder

Device Utilization

IO (Input/Output) refers to the physical FPGA pins used for communication with external devices, tracking how many of these pins are utilized for inputs and outputs. Utilization, on the other hand, measures the internal FPGA resource consumption, such as lookup tables (LUTs), flip-flops, block RAM (BRAM), and DSP slices. Together, these concepts provide a comprehensive view of FPGA design efficiency, as shown in figure 4.

Power Estimation

Figure 10 shows estimation of static power, dynamic power (power required when VHDL program is burned in FPGA) and other related power datas of a full adder circuit.

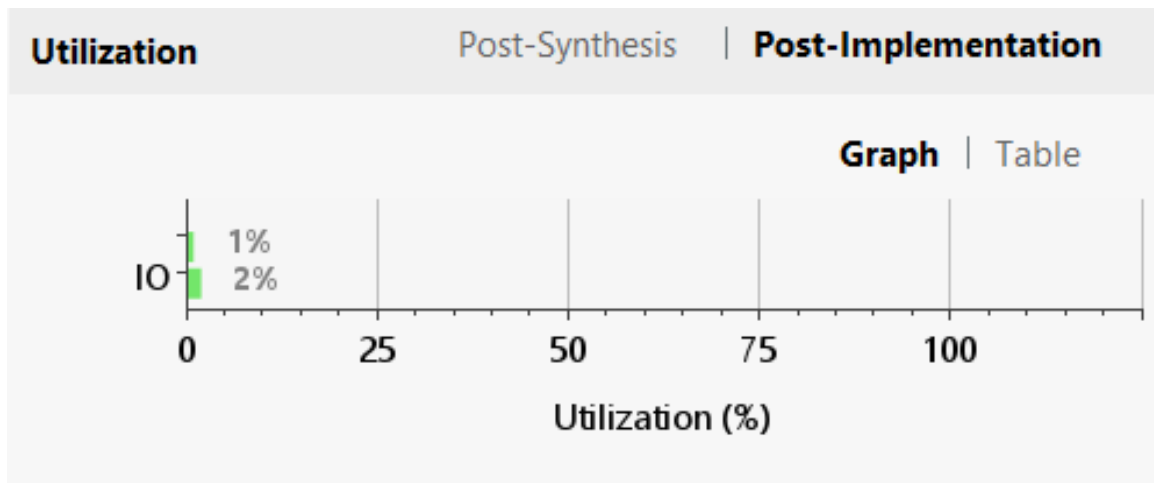


Figure 9: Device Utilization of full adder

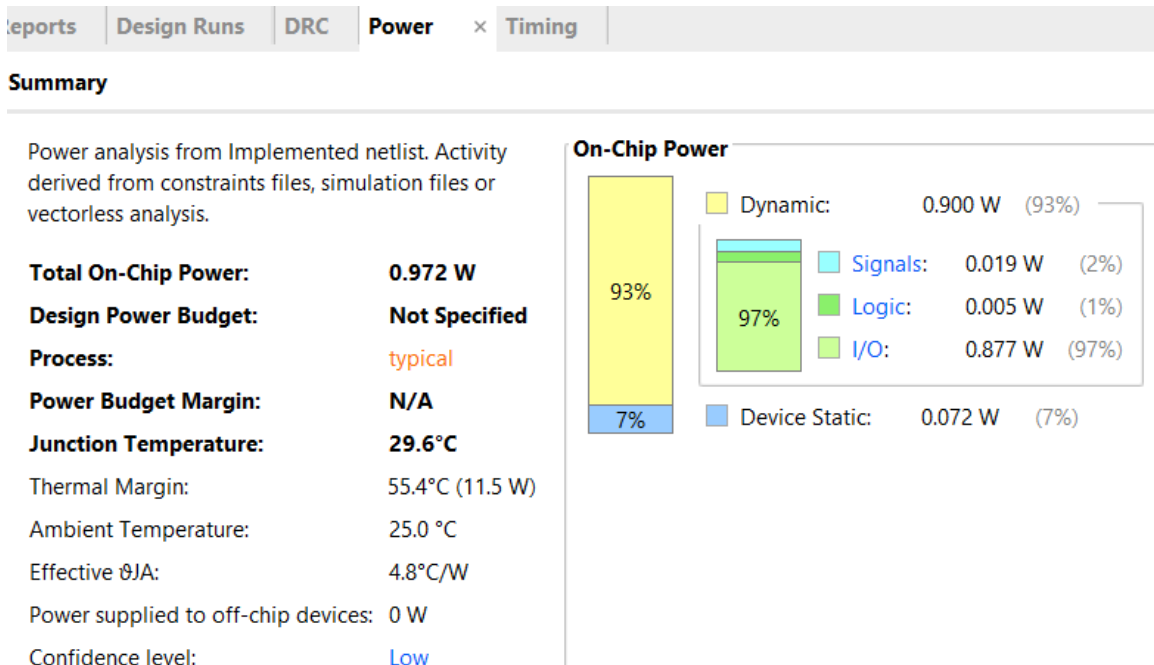


Figure 10: Power Estimation of full adder

2.3 Half Subtractor

VHDL code

```
use IEEE.STD_LOGIC_1164.ALL;
-- designing half subtractor circuit
entity Half_Subtractor is
    Port ( A : in STD_LOGIC;
          B : in STD_LOGIC;
          Diff : out STD_LOGIC;
          Borrow : out STD_LOGIC);
end Half_Subtractor;

architecture Behavioral of Half_Subtractor is

begin
    Diff <= A xor B;
    Borrow <= (not A) and B;

end Behavioral;
```

RTL Schematic Diagram

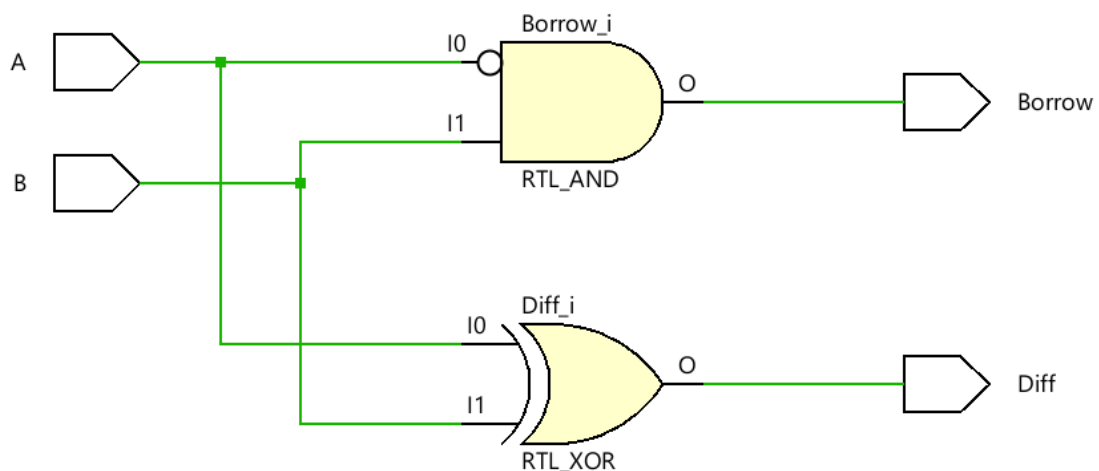


Figure 11: RTL Schematic Diagram of half subtractor

RTL design is used to describe the behavior of the half subtractor as shown in figure 11

Synthesis Schematic Diagram

Schematic design is used to describe the implementation of the half subtractor circuit as shown in figure 12

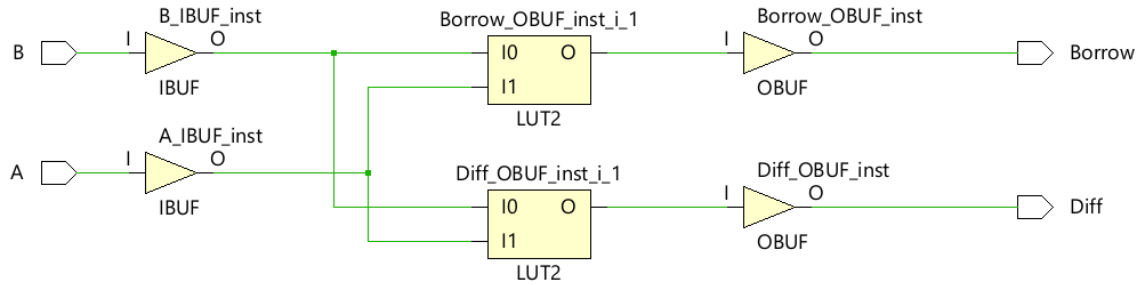


Figure 12: Synthesis Schematic Diagram of half subtractor

Timing diagram

Timing diagram test is done to check whether the VHDL program and schematic diagram is valid for the given truth table of half subtractor.

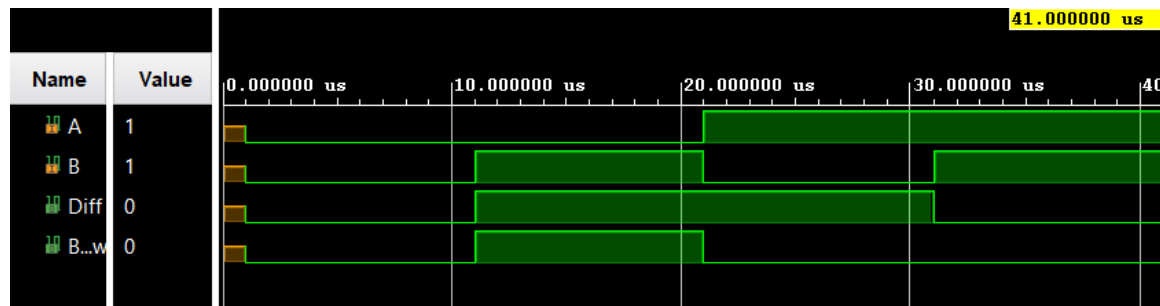


Figure 13: Timing Diagram of half subtractor

Device Utilization

IO (Input/Output) refers to the physical FPGA pins used for communication with external devices, tracking how many of these pins are utilized for inputs and outputs. Utilization, on the other hand, measures the internal FPGA resource consumption, such as lookup tables (LUTs), flip-flops, block RAM (BRAM), and DSP slices. Together, these concepts provide a comprehensive view of FPGA design efficiency, as shown in figure 14.

Power Estimation

Figure 15 shows estimation of static power (no change in input and output), dynamic power (power required when VHDL program is burned in FPGA) and other related power data of a half subtractor circuit.

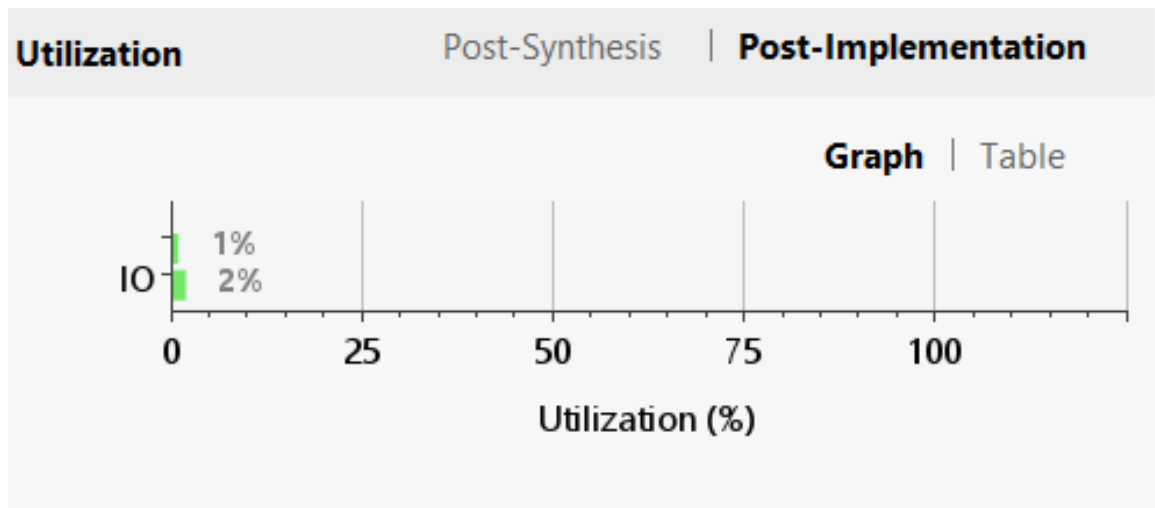


Figure 14: Device Utilization of half subtractor

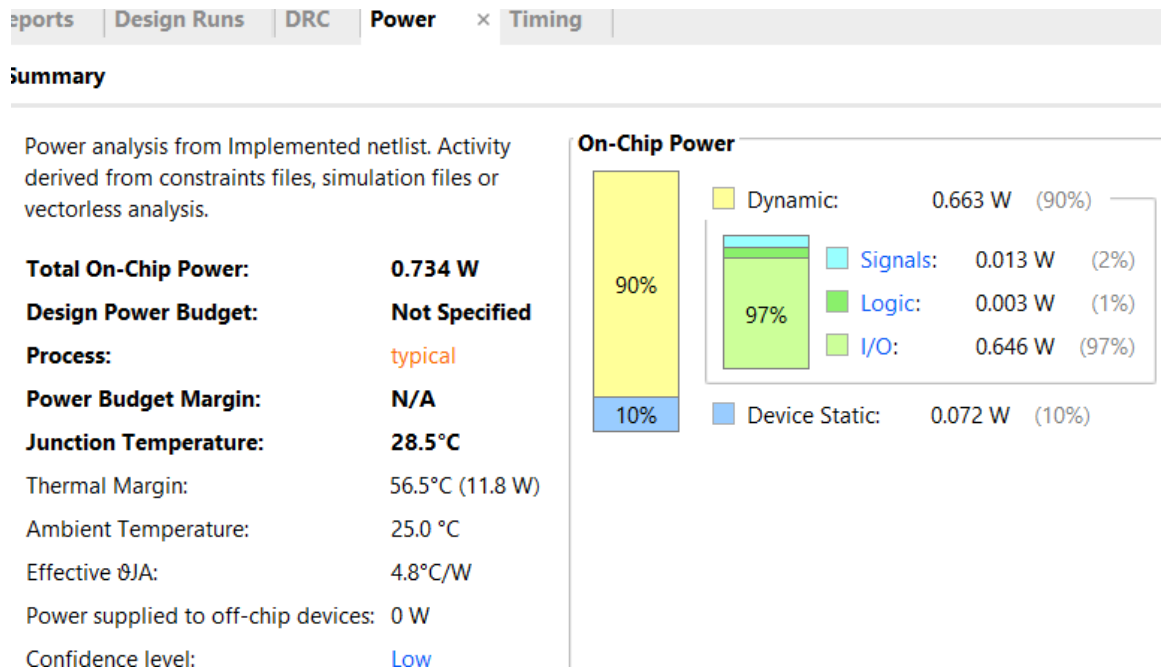


Figure 15: Power Estimation of half subtractor

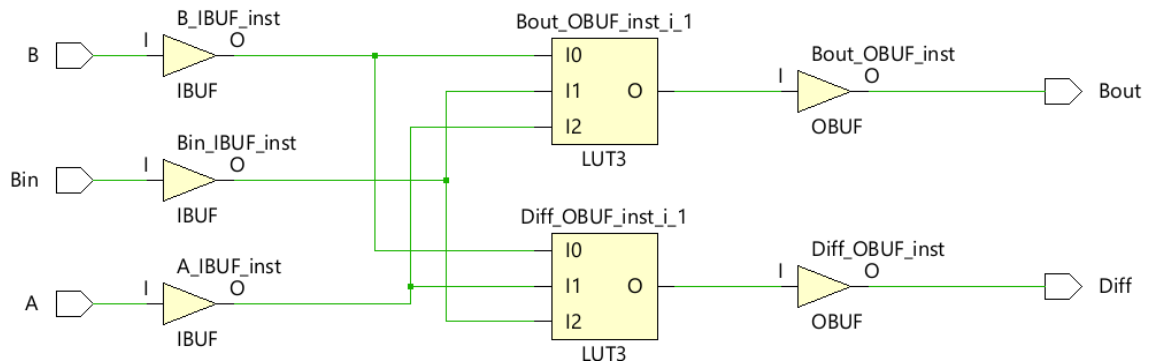


Figure 17: Synthesis Schematic Diagram of full subtractor

Timing diagram

Timing diagram test is done to check whether the VHDL program and schematic diagram is valid for the given truth table of full subtractor.

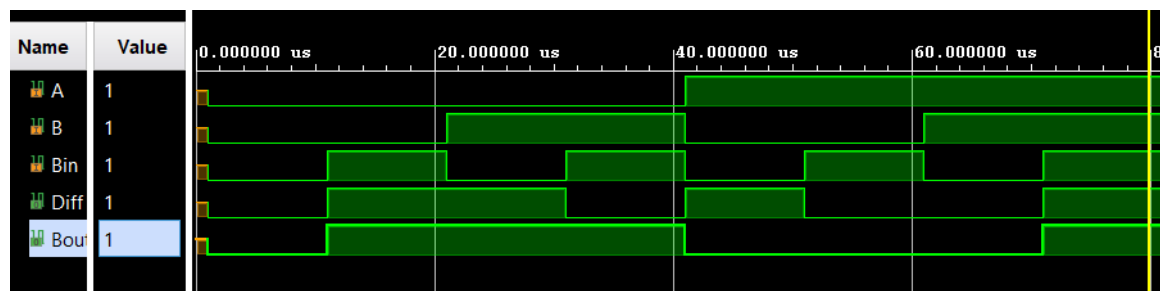


Figure 18: Timing Diagram of full subtractor

Device Utilization

IO (Input/Output) refers to the physical FPGA pins used for communication with external devices, tracking how many of these pins are utilized for inputs and outputs. Utilization, on the other hand, measures the internal FPGA resource consumption, such as lookup tables (LUTs), flip-flops, block RAM (BRAM), and DSP slices. Together, these concepts provide a comprehensive view of FPGA design efficiency, as shown in figure 19.

Power Estimation

Figure 20 shows estimation of static power, dynamic power (power required when VHDL program is burned in FPGA) and other related power datas of a full subtractor circuit.

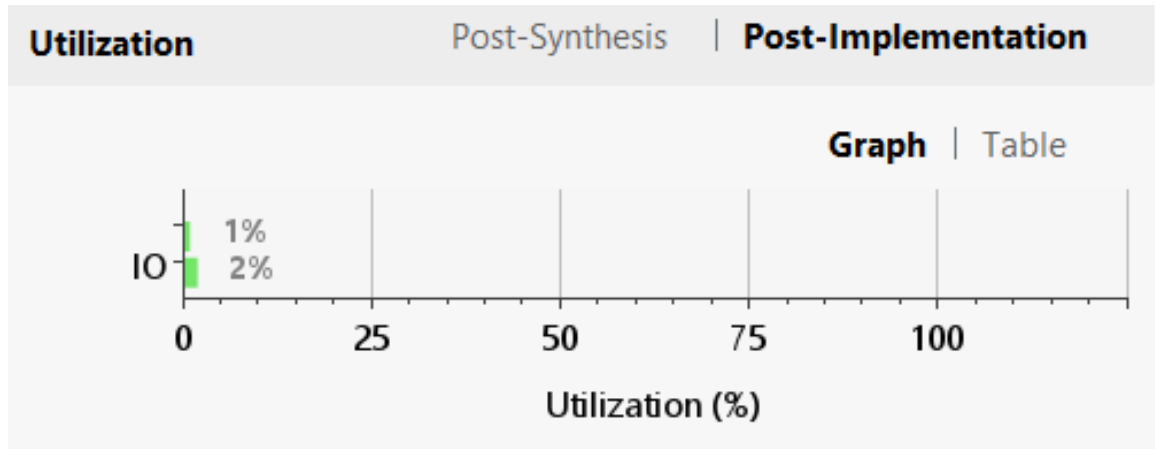


Figure 19: Device Utilization of full subtractor

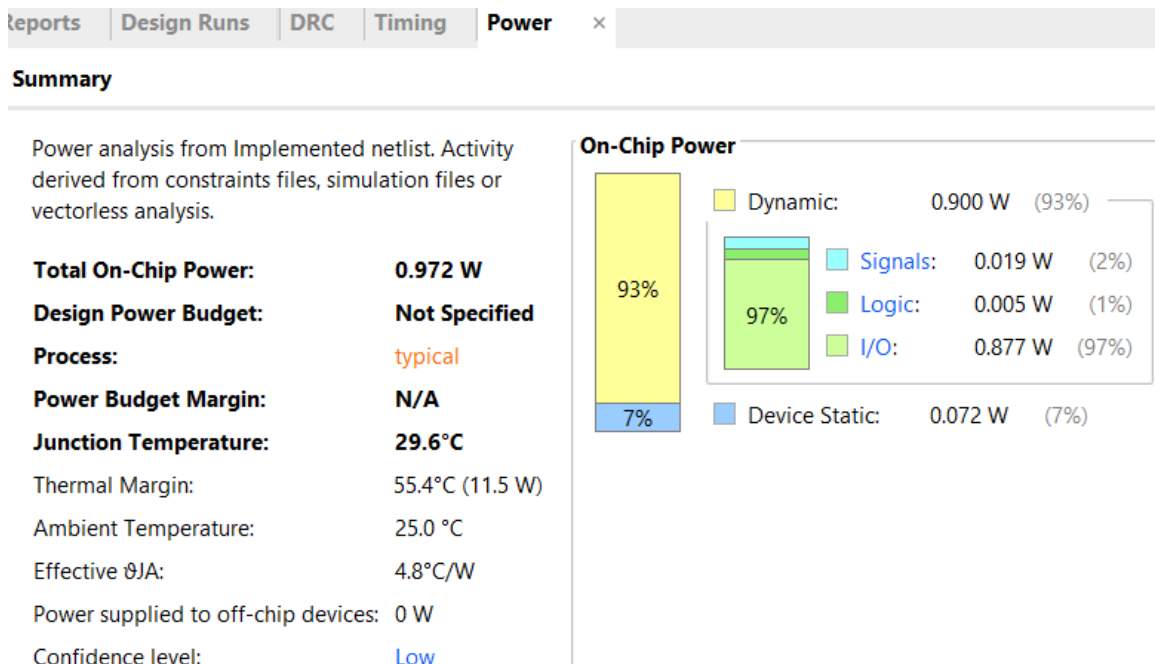


Figure 20: Power Estimation of full subtractor

3 Conclusion

In this lab, we implemented the VHDL code for a half adder, full adder, half subtractor, and full subtractor on the Atrix-7 FPGA (part: xc7a50tcs324-1, package: csg324) using Vivado software. Each design was synthesized, simulated, and implemented, followed by hardware testing, which confirmed their correct functionality.

The process began with simulation to verify the timing diagrams, followed by linting to check for syntax errors. Afterward, the designs were synthesized into a hardware logic circuit, and during the implementation phase, power and timing characteristics were analyzed, displayed and studied. A bitstream file was generated, and the necessary pins were assigned for I/O connections. After successful verification, the design was programmed onto the FPGA board and the desired outputs were obtained.